

太原理工大学先进计算机系统实验室（ACSL）

正式学员第三次学习路线

学习情况：持续进行Verilog实践练习和LCTHW实践，学习了更为底层的编译流程，最终用C语言完成较为简单的指令集模拟器。

学习目标：结束Verilog与LCTHW的实践，更深入学习编译流程，最终用C语言完成支持8条指令的minirvEMU。

数字设计

我们在寒假的时候学习过状态机的概念，无非就是电路在不同的状态转移罢了，本周我们需要通过Verilog完成状态机的设计。这是数字设计相当困难的一章，思路理不清楚很有可能坐一天也想不出来，这时候就可以试着**画图**，理清楚思路再写这部分代码效果会好很多哦！并且这一部分是有固定套路的，大家在做了很多道相关题之后，**做笔记归纳总结**，就可以悟出状态机的构建流程了。

同时有的题目网站给出的示例代码并不是特别好，有的时候**完全推翻重新写**会比按照网站给出的示例写快不少，且思路更加清晰。

Verilog

本周你需要完成HDLBits中的如下任务，这里如果遇到卡住很久的题可以先跳过，同时有限状态机部分题目有很多重复，大家对这里有自己的方法和理解即可，**可以跳过重复性较高的题!!!**提交你的截图即可。（在这里推荐一个浏览器插件：**沉浸式翻译**，如果看不懂的话，就用这个插件配合学习吧！）

- Serial receiver ○
- Serial receiver and datapath ○
- Serial receiver with parity checking ○
- Sequence recognition ○
- Q8: Design a Mealy FSM ○
- Q5a: Serial two's complementer (Moore FSM) ○
- Q5b: Serial two's complementer (Mealy FSM) ○
- Q3a: FSM ○
- Q3b: FSM ○
- Q3c: FSM logic ○
- Q6b: FSM next-state logic ○
- Q6c: FSM one-hot next-state logic ○
- Q6: FSM ○
- Q2a: FSM ○
- Q2b: One-hot FSM equations ○
- Q2a: FSM ○
- Q2b: Another FSM ○

! 我们培养的是硬件思维，需要头脑中先有电路再下手写代码，这也是为什么我们需要先学习使用Logisim搭建数字电路，再来学习数字设计，虽然我们后面不再使用Logisim进行处理器设计，但Logisim的使用经验应该已经帮助你建立了"电路思维"：数字电路设计只做两件事，"实例化"和"连线"。你接下来使用HDL来设计数字电路时，头脑中也需要将HDL代码和Logisim的使用经验建立关联：你只不过是换了一种方式来设计电路，但本质上还是在进行"实例化"和"连线"的工作，因此你应该能根据你编写的代码想象到电路的逻辑结构，**要记住Verilog的本质是硬件描述语言而不是传统的编程语言。**

Learn C the hard way

Makefile

大家在Learn C the hard way的第2篇的时候其实就已经接触过Makefile了，但那时只是简单的理解为一键编译运行，今天就要进阶到Makefile的高阶用法了。

Makefile是我们后期很重要的部分，一定要学好。它是项目构建的必需语言，利用Makefile我们可以把很复杂的项目一键编译运行，是未来我们做大型项目的必备知识，我们这边提供一些学习理论知识的途径。

- [20分钟Makefile光速入门教程_哔哩哔哩_bilibili](#)（在最后介绍了CMake，是一种跨平台的更现代的构建系统）
- [跟我一起写Makefile — 跟我一起写Makefile 1.0 文档](#)（推荐）
- <https://seisman.github.io/how-to-write-makefile/Makefile.pdf>
- <https://www.gnu.org/software/make/manual/make.html>（官方文档，RTFM）

Makefile作为一门语言，而且是很特殊的语言，学习的**最好方法**是**多看大型项目中的Makefile是怎么写的，它是怎么把各组件链接起来的**，这样才能把Makefile学个差不多，只学理论很难学明白。而本次学习路线中的Learn C the hard way刚好有一个很适合学习Makefile的练习。

也因此现阶段的Makefile学习到能够看懂Learn C the hard way第28篇给出的示例并且能够自己修改的程度即可。

Learn C the hard way

本周完成其中的第**28~33**章，其中遇到卡了很久的题目可以先跳过，待有思路再重新思考。本次这部分作业的**附加题不要求实现**，只需要实现此部分内容中的必做部分即：如，**28~31**节理解框架构建和运行过程；**32~33**学会数据结构和算法的具体实现过程。

【Learn C the hard way】：

<https://wizardforcel.gitbooks.io/lcthw/content/preface.html>

将你完成的所有练习放入一个名为 `lcthw` 的文件夹，并将该文件夹放入作业提交文件夹中。

! 本次的学习很难，C语言部分迎来多种知识杂糅运用——你们会学到编译链接等偏底层的知识，又要学习数据结构的具体实现过程；你可能会在这一部分的学习过程中处于懵逼状态，看不懂代码，看不懂他的实现目的；这时，我推荐大家问AI，AI可以帮你**总结出这一部分的知识点**，也可以帮你**理清实现目的**，但是**不要过度相信AI，可疑的部分一定要去上网查，或者问同学或学长**。

从源代码到可执行文件（2）

汇编——从字符到纯粹的二进制

编译的结果是汇编代码，是纯粹的字符表示。而到了后面的汇编这一步，我们的程序才算变成了机器可以“读懂”的东西。用更专业的话来说，汇编就是将汇编代码转变成指令的二进制编码。

汇编器在这里要做的事情很简单，就是查阅指令集手册，并将汇编代码中的文本指令逐条翻译成相应的二进制编码，最终形成目标文件（object file，Linux中以.o为后缀）。

阅读目标文件

编写一个 `hello world` 程序，使用 `-S` 选项得到其汇编代码，再使用 `-c` 选项得到目标文件，最后通过 `riscv64-linux-gnu-objdump -d <object file>` 命令阅读目标文件的反汇编结果，观察反汇编结果与编译得到的汇编代码有什么异同。

最后，将你的 `.c` 文件、`.s` 文件、`.o` 文件与最终反汇编的结果（重命名为 `obj.S`）放入作业提交文件夹中。

链接——成为真正的程序

仅仅会“读懂”并不足以满足机器运行程序的条件，我们还差最后一步——链接（Link）。在链接完成以前，我们的程序并不知道如何调用 `printf` 等外部函数。

链接的工作是将多个目标文件合并成最终的可执行文件，我们可以通过如下命令阅读最终生成的可执行文件都多了些什么东西：

代码块

```
1 $ riscv64-linux-gnu-gcc hello.c -o hello
2 $ riscv64-linux-gnu-objdump -d hello
```

和链接前的目标文件（`hello.o`）相比，链接后的可执行文件中多出了不少内容。它们就是链接器带来的相关代码。

阅读可执行文件的反汇编结果

执行以上代码，大致阅读一遍反汇编结果后，将反汇编结果（重命名为 `out.S`）放入作业提交文件夹中。

运行时环境

阅读完反汇编结果后，你发现多出来的很多函数你从来都没有见到过，这些函数都从哪里来的呢？我们可以运行如下命令：

代码块

```
1 $ riscv64-linux-gnu-gcc -c hello.c
2 $ riscv64-linux-gnu-gcc -v hello.o
```

在日志末尾可以看到链接相关的命令，在该命令中还包含了若干命名形如 `crt*.o` 的目标文件。此处的 `crt` 是 `C runtime` 的缩写，表示C程序的运行时环境。

链接过程会将目标文件（`hello.o`），和已有的C程序运行时环境相关的目标文件进行合并，最终生成可执行文件。可以预见，可执行文件之所以要包含这些和运行时环境相关的目标文件，是为了向可执行文件的执行提供必要的支持。

你在其中还可以找到我们曾在[国太原理工大学先进计算机系统实验室（ACSL）适应期学员第四次学习路线](#)中提到过的 `_start` 函数，`_start` 便是运行时环境中的一部分。

在观看完汇编结果后你是否有过疑问：为何预处理结果这么长而汇编代码却如此短？为什么我们能找到的只有 `puts@plt` 或 `printf@plt` 而不是 `printf`？它们涉及编译器优化和动态库调用的知识，或许在可能存在的下一期我们能够继续学习它们。感兴趣的同学可以通过 `STFW` 和 `RTFM` 进一步学习。

未完待续

下一篇 从源代码到可执行文件（3）开放时间待定

MinirvEMU

MinirvEMU

阅读[一生一芯官方讲义E4讲义minirvEMU部分](#)并用C语言代码编写 `minirvEMU` 模拟器。

你可以在终端输入 `git clone https://github.com/Hoan-Meirin/ACSL-minirvEMU.git` 命令获取我们准备的框架代码，（当然，这份框架代码不仅需要魔法上网而且理解起来也不容易，你也可以选择自己从0开始写，我们很支持的）。

注：我们的框架代码已经准备好了 `Makefile`，你需要 `RTFSC` 理解框架代码的运行逻辑。

完成后将你的文件或文件夹命名为 `minirvEMU`，并放入作业提交文件夹中。

作业提交



1. HDLBits的截图命名为 `Verilog` 并放到 `姓名-专业班级-Great-16` 文件夹中。
2. `lcthw` 文件夹也放在上述文件夹里。
3. 其他在讲义中提及需要提交的文件按要求进行整理提交。
4. 如果你学有余力完成了下面的拔高内容，则把文件夹重命名，格式为 `姓名-专业班级-NewStar-16`。
5. 将你的作业压缩为zip格式并提交到[作业提交表单](#)。

拔高内容

一生一芯课程PA

PA是我们后续学习中非常重要的一部分内容，目前我们已经把PA0相关的基础知识进行了补全，大家**可以去尝试PA0的相关内容学习**，而后面的PA1是一生一芯预学习答辩的主要内容之一，如果你想挑战自己，现在去尝试PA1也是没问题的。



PA0

<https://ysyx.oscc.cc/docs/ics-pa/PA0.html>

! 当你发现如下提醒时，阅读该讲义：<https://ysyx.oscc.cc/docs/2407/e/3.html>获取属于一生一芯的代码框架

! 参加"一生一芯"的同学, 请参考"一生一芯"讲义获取代码链接

如果你参加"一生一芯", 请勿使用下面的代码链接. 此外, PA讲义中关于作业提交的要求, "一生一芯"的同学可以全部忽略, 但需要关注"一生一芯"讲义中的提交要求.

一生一芯E4阶段讲义

进度较快的你已经有足够的实力与ysyx的官方讲义接轨了，完成ysyxE阶段的学习之后即可参加项目组的预学习答辩，答辩通过即成为ysyx项目组的正式成员，并且收到一份来自项目组的入学大礼包！



E4

E4阶段的讲义看起来非常长，但细细看来，你会发现很多似曾相识的东西，这其实就是我们之前讲义埋下的伏笔了，

<https://ysyx.oscc.cc/docs/2407/e/4.html>

阅读并跟着讲义完成其中的实操，并完成 `带图形显示功能minirvEMU`。



在 `支持GUI输入输出的程序` 之前，请先完成PA0并获取一生一芯的框架代码

一生一芯E5阶段讲义

什么，你已经看到这里了，无论是你写到这里了还是好奇你们要做到的极限，这都说明你有能力尝试更加具备挑战性的任务！

我们都知道RTL代码是一种硬件描述语言，但是想让你写在文件中的代码真正模拟出硬件的行为，我们还需要搭建仿真环境来验证自己代码的正确性。

E5

一生一芯E5讲义

阅读并跟着讲义完成其中的实操任务，并最终搭建Verilator仿真环境，将流水灯接入NVBoard。

E5最终目标是完成 `南京大学的数字电路实验` 与 `用RTL实现简单处理器` 你只需要朝着这个目标继续往下写就可以了，中间涉及到很多与ic后端相关的知识了解即可（PDK和标准单元库 到 若干代码风格和规范之间（不包括）的内容了解即可）。